algorithm. As a matter of facts, there are cases where recursion is the right choice, usually because either the very nature of the problem or its definition are recursive. Usually it's more a matter of clarity, of writing clean code, than it is of performance. In a couple of sections, however, we'll see how *tail recursion* combines both (at least in modern programming languages).

But for now, let's take a look at an elegant solution for the pre-order traversal of a binary tree:

Listing E.3 Binary tree preorder traversal

```
function preorder(node) {
  if (node === undefined || node === null) {
    console.log("leaf");
  } else {
    console.log(node.value)
    preorder(node.left);
#1
    preorder(node.right);                                    #2
  }
```

#1 Recursive traversal of the left child
#2 Recursive traversal of the right child

Preorder traversal is a way of enumerating the keys stored in a tree. It creates a list starting with the key stored in the root, and then it recursively traverses its children's subtrees: when it gets to any node, it first adds the node's key to the list, then goes to the children